НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»
Інститут прикладного системного аналізу
Кафедра системного проектування

ЛАБОРАТОРНА РОБОТА №3

з курсу: „Паралельні обчислення”

виконав: студент 3го курсу
групи ДА-11
Сидько Нікіта

КИЇВ - 2024

Мета роботи: Розглянути поняття атомарності, навчитися працювати з атомарними змінними, а також ознайомитися з підходом написання паралельного коду без блокування.

Завдання

1. Ознайомитися з визначенням: атомарна змінна, атомарна операція, неблокуючий алгоритм. Ознайомитися з деталями атомарності в обраній мові програмування.

2. Надати в протоколі роботи опис того, як саме досягається справжня атомарність операцій в обраній студентом мові програмування.

3. Виконати завдання за варіантом без використанням паралелізації. Заміряти час виконання завдання.

4. Виконати завдання за варіантом з використанням блокуючих примітивів синхронізації. Заміряти час виконання завдання.

5. Виконати завдання за варіантом з використанням атомарних змінних та CAS\CMPXCHNG операцій (більш високорівнені функції, що абстрагують дані операції, не приймаються для використання в даній роботі). Заміряти час виконання завдання.

6. Повторити пункти 2 – 4 з використанням різної розмірності даних та фіксованою кількістю потоків виконання.

7. В протокол занести отримані для пункту 5 результати у вигляді графіків залежності часу від кількості даних, надати порівняльний аналіз блокуючого та неблокуючого алгоритму з використання атомарних операцій.

8. Надати висновок, що повинен містити аналіз отриманих результатів.

Варіант

Знайти кількість елементів, кратних 19, і найменший такий елемент

1. Ознайомитися з визначенням: атомарна змінна, атомарна операція, неблокуючий алгоритм. Ознайомитися з деталями атомарності в обраній мові програмування.

Атомарність – це властивість, що визначена для операцій, і говорить про те, що операція може мати лише один з двох станів – виконана, або не виконана, не допускаючи втручання інших процесів або потоків у середині виконання операції. Це забезпечує коректність результатів операцій, коли вони виконуються в паралельних середовищах. Більш детально, коли операція є атомарною, це означає, що вона або виконалася повністю, або ще не виконувалася взагалі, і будь-які інші проміжні стани відсутні з точки зору стороннього наглядача (в даному випадку – інших потоків). Атомарність запобігає ситуаціям, коли стан операнда (змінної, над або за допомогою якої проводиться операція, чи системи в цілому) може змінитись під час виконання операції, порушуючи очікувану по слідовність дій та приводячи до некоректних результатів.

Атомарні змінні – це спеціальні типи даних, призначені для коректного використання в паралельних середовищах. Вони гарантують атомарність операцій, тобто виконання окремих операцій без переривань, забезпечуючи коректність результатів, навіть при одночасному доступі декількох потоків.

Атомарні змінні дозволяють виконувати ряд атомарних операцій, таких як:

- Зчитування та запис значень;
- Операції порівняння та обміну значень (CAS або ж CMPXCHG);
- Інкремент і декремент;
- Арифметичні операції додавання, віднімання, множення та ділення.

Використання атомарних змінних дозволяє забезпечити синхронізацію доступу до спільних ресурсів між різними потоками без використання традиційних (блокуючих) механізмів синхронізації, таких як м’ютекс або семафори. Це може покращити продуктивність програми, зменшуючи навантаження на систему і ризик виникнення взаємного блокування або гонки за ресурсами.

Атомарна операція — це комп'ютерна операція, яка виконується повністю без можливості переривання або знаходження у проміжному стані для інших потоків. Атомарні операції важливі у багатопотоковому програмуванні, оскільки вони дозволяють уникнути гонок даних та інших проблем з синхронізацією. У C++, атомарні операції можна реалізувати за допомогою класу std::atomic, який забезпечує атомарність операцій навіть на змінних простих типів, як-от int або long.

Неблокуючий алгоритм — це алгоритм, що забезпечує синхронізацію між потоками без використання блокуючих примітивів, таких як м’ютекс. В основі всіх неблокуючих алгоритмів лежать або відсутність використання синхронізації, або ж атомарні операції та змінні, які дозволяють виконувати операції над спільними ресурсами, не призупиняючи роботу інших потоків виконання та не приводячи до операції зміни контексту виконання

2. Надати в протоколі роботи опис того, як саме досягається справжня атомарність операцій в обраній студентом мові програмування.

C++ визначає набір атомарних типів для базових типів даних, таких як int, long, bool, тощо. Ці типи обгортані у шаблонний клас std::atomic<T>, де T — це тип даних. Наприклад, std::atomic<int> або std::atomic<bool>. Ці типи гарантують, що читання та запис до змінної виконуються атомарно. Бібліотека <atomic> надає ряд функцій та операторів для виконання атомарних операцій на змінних. Ці операції включають зчитування та запис (**load та store**) - атомарно читають або записують значення. Модифікація та збереження (**fetch-and-add, fetch-and-sub** - атомарно модифікують значення та повертають попереднє значення. Порівняння та обмін (**compare-and-swap**) - порівнюють поточне значення змінної з заданим значенням і, якщо вони рівні, атомарно замінюють на нове значення.

Атомарні операції в C++ підтримують різні меморіальні моделі, які дозволяють програмістам вказати порядок видимості операцій в многопоточному середовищі. Це дозволяє оптимізувати виконання програми, забезпечуючи при цьому необхідні гарантії синхронізації. Наприклад, меморіальні моделі std::memory_order_relaxed, std::memory_order_acquire, std::memory_order_release та інші визначають різні рівні строгості порядку виконання операцій.

Атомарні операції у C++ здійснювані на рівні компілятора та апаратного забезпечення. Компілятор генерує інструкції, які використовують апаратні можливості процесора для виконання атомарних операцій, забезпечуючи, що вони виконуються без перервання.

3. Виконати завдання за варіантом без використанням паралелізації. Заміряти час виконання завдання.

Для цього написано окрему функцію яка виглядає наступним чином :

```
void SingleThreadSolution(int** matrix, int size) {
    auto start = std::chrono::high_resolution_clock::now();
    int count = 0;
    int min = INT_MAX;

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            int num = matrix[i][j];
            if (num % DIVISOR == 0) {
                count++;
                if (num < min) {
                    min = num;
                }
            }
        }
    }

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end - start;

    Print(count, min);
    std::cout << "Пройдений час: " << elapsed.count() << " секунд" << std::endl;
}
```

Результат роботи функції :

Суть методу полягає у переборі всіх чисел матриці, заміні на найменше число та підрахунку кількості кратних чисел.

```
Без використання паралелезації:
Кількість чисел що ділятся на 19: 1314021
Найменше число: 19
Пройдений час: 0.124006 секунд
```

4. Виконати завдання за варіантом з використанням блокуючих примітивів синхронізації. Заміряти час виконання завдання.

Для виконання данного пункту будемо використовувати mutex. Для данного пункту було написано два види функцій (оптимізована та не оптимізована) останню було зроблено для порівняння графіків блокуючих примітивів та атоміків.

```
// не опт
void MutexCompute(int** matrix, int size, int start, int end, int& count, int& min, std::mutex& mtx) {
    for (int i = start; i < end; i++) {
        for (int j = 0; j < size; j++) {
            if (matrix[i][j] % DIVISOR == 0)
            {
                mtx.lock();
                count++;
                if (matrix[i][j] < min)
                {
                    min = matrix[i][j];
                }
                mtx.unlock();
            }
        }
    }
}
```

```cpp
// опт

void MutexCompute(int** matrix, int size, int start, int end, int& count, int& min, std::mutex& mtx) {
    int local_count = 0;
    int local_min = INT_MAX;
    for (int i = start; i < end; i++) {
        for (int j = 0; j < size; j++) {
            if (matrix[i][j] % DIVISOR == 0) {
                local_count++;
                if (matrix[i][j] < local_min) {
                    local_min = matrix[i][j];
                }
            }
        }
    }
    mtx.lock();
    count += local_count;
    if (local_min < min) {
        min = local_min;
    }
    mtx.unlock();
}
```

```cpp
void ParallelComputeMutex(int** matrix, int size, int num_threads) {
    std::vector<std::thread> threads;
    int segment = size / num_threads;
    int count = 0, min = INT_MAX;
    std::mutex mtx;


    auto start_time = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < num_threads; i++) {
        int start_index = i * segment;
        int end_index;
        if (i == num_threads - 1)
        {
            end_index = size;
        }
        else
        {
            end_index = start_index + segment;
        }
        threads.emplace_back(MutexCompute, matrix, size, start_index, end_index, std::ref(count), std::ref(min), std::ref(mtx));
    }

    for (auto& thread : threads) {
        thread.join();
    }
    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end_time - start_time;

    Print(count, min);
    std::cout << "Час виконання з мютексами: " << elapsed.count() << " секунд." << std::endl;
}
```

Суть методу полягає у наступному:

MutexCompute

Кожен потік обробляє визначений діапазон рядків в матриці (start до end). Це дозволяє паралельно обробляти великі дані, знижуючи загальний час виконання. Для синхронізації доступу до спільних змінних (count та min) використовується мютекс. Коли потік знаходить число, кратне заданому дільнику (DIVISOR), він блокує мютекс для безпечного оновлення спільних змінних. Кожен раз, коли знаходиться число, що ділиться на DIVISOR, змінна count інкрементується.

Якщо знайдене число також є меншим за поточне мінімальне значення, то min оновлюється. Після оновлення спільних змінних мютекс розблоковується для надання доступу іншим потокам.

ParallelComputeMutex
Матриця розділяється на сегменти, розраховані відповідно до кількості потоків. Кожен потік отримує сегмент матриці для обробки. Для кожного сегмента створюється потік, який виконує MutexCompute з відповідними параметрами початку та кінця сегмента. Після старту всі потоки працюють паралельно, і програма чекає (join) завершення всіх потоків перед продовженням.

Оптимізація:
Суть полягає у глобальних змінніх, які впливають на кількість звертань, за їх відсутності. Тобто оптимізований метод порівняно з неоптимізованим у використанні мютексів полягає в зменшенні кількості операцій блокування та розблокування мютекса, що значно підвищує ефективність паралельного виконання.

У неоптимізованому методі мютекс блокується і розблокується в кожній ітерації циклу, де відбувається зміна спільних змінних (count і min). Оптимізований метод зводить до мінімуму використання мютексів, використовуючи їх лише один раз після обробки всієї частини матриці, яка призначена для конкретного потоку

5. Виконати завдання за варіантом з використанням атомарних змінних та CAS\CMPXCHNG операцій (більш високорівнені функції, що абстрагують дані операції, не приймаються для використання в даній роботі). Заміряти час виконання завдання.

```
void AtomicCompute(int** matrix, int size, int start, int end, std::atomic<int>& count, std::atomic<int>& minimum) {
    for (int i = start; i < end; i++) {
        for (int j = 0; j < size; j++) {
            if (matrix[i][j] % DIVISOR == 0)
            {
                count.fetch_add(1);
                int current_min = minimum.load();
                do {
                    if (matrix[i][j] >= current_min) {
                        break;
                    }
                } while (!minimum.compare_exchange_weak(current_min, matrix[i][j]));
            }
        }
    }
}
```

```cpp
void AtomicCompute(int** matrix, int size, int start, int end, std::atomic<int>& count, std::atomic<int>& minimum) {
    int local_count = 0;

    for (int i = start; i < end; i++) {
        for (int j = 0; j < size; j++) {
            if (matrix[i][j] % DIVISOR == 0) {
                local_count++;
                int current_min = minimum.load();

                do {
                    if (matrix[i][j] >= current_min) {
                        break;
                    }
                } while (!minimum.compare_exchange_weak(current_min, matrix[i][j]));
            }
        }
    }
    count.fetch_add(local_count);
}
```

```cpp
void ParallelComputeAtomic(int** matrix, int size, int num_threads) {
    std::vector<std::thread> threads;
    int segment = size / num_threads;
    std::atomic<int> count(0), minimum(INT_MAX);


    auto start_time = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < num_threads; i++) {
        int start_index = i * segment;
        int end_index;
        if (i == num_threads - 1)
        {
            end_index = size;
        }
        else
        {
            end_index = start_index + segment;
        }
        threads.emplace_back(AtomicCompute, matrix, size, start_index, end_index, std::ref(count), std::ref(minimum));
    }

    for (auto& thread : threads) {
        thread.join();
    }
    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end_time - start_time;

    Print(count.load(), minimum.load());
    std::cout << "Час виконання з атоміками: " << elapsed.count() << " секунд." << std::endl;
}
```

std::atomic<int> count і std::atomic<int> minimum є атомарними змінними. Це означає, що операції з цими змінними виконуються атомарно, тобто без переривань іншими потоками.

Для кожного елементу матриці перевіряється умова дільності на DIVISOR. Якщо елемент задовольняє умову, то виконуються наступні дії: count.fetch_add(1): Атомарно збільшується лічильник кількості елементів, кратних DIVISOR.

minimum.compare_exchange_weak(current_min,matrix[i][j]) атомарно оновлює мінімальне значення, якщо поточний елемент матриці менший за збережене мінімальне значення. Ця операція виконується в циклі do-while, що гарантує атомарність оновлення (якщо під час оновлення значення було змінено іншим потоком, операція повторюється з новим актуальним значенням)

Не оптимізований:

```
Введіть розмір матриці: 2500
Введіть кількість потоків: 4

Без використання паралелезації:
Кількість чисел що ділятся на 19: 329322
Найменше число: 19
Пройдений час: 0.0406176 секунд

З використанням мютекса:
Кількість чисел що ділятся на 19: 329322
Найменше число: 19
Час виконання з мютексами: 0.0776612 секунд.

З використанням атоманрих змінних:
Кількість чисел що ділятся на 19: 329322
Найменше число: 19
Час виконання з атоміками: 0.0298921 секунд.
```

Оптимізований :

```
Введіть розмір матриці: 2500
Введіть кількість потоків: 4

Без використання паралелезації:
Кількість чисел що ділятся на 19: 329052
Найменше число: 19
Пройдений час: 0.0310992 секунд

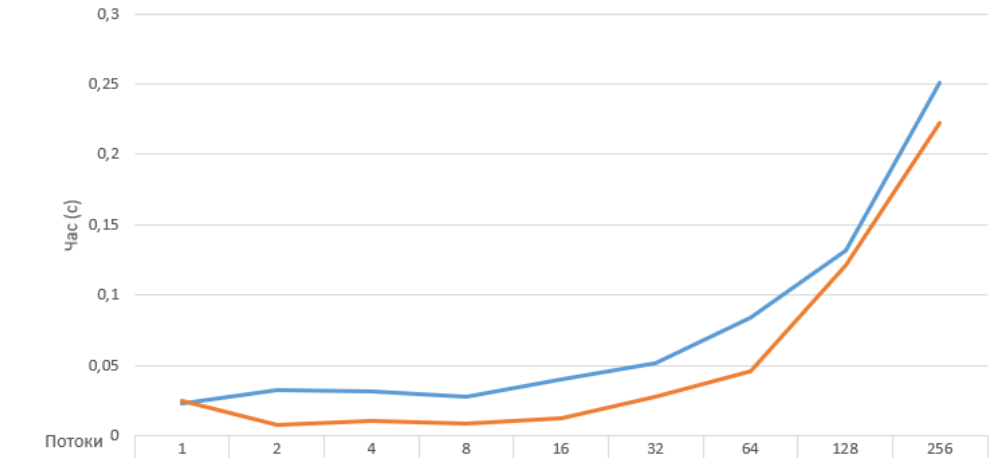
З використанням мютекса:
Кількість чисел що ділятся на 19: 329052
Найменше число: 19
Час виконання з мютексами: 0.0318727 секунд.

З використанням атоманрих змінних:
Кількість чисел що ділятся на 19: 329052
Найменше число: 19
Час виконання з атоміками: 0.0255613 секунд.
```

6 - 7. Повторити пункти 2 – 4 з використанням різної розмірності даних та фіксованою кількістю потоків виконання. Графіки

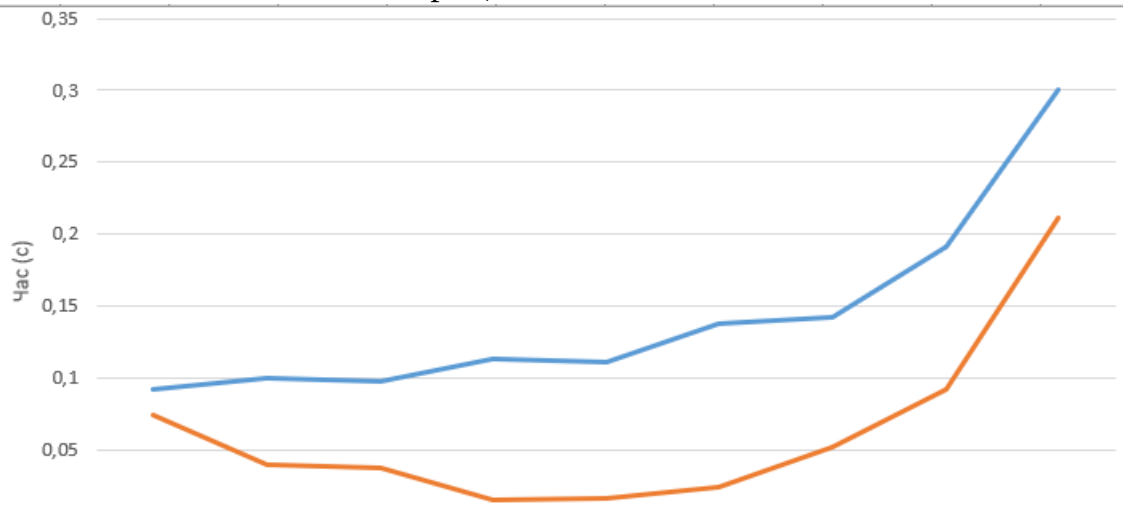
Блокування(не опт)		Кількість потоків	1	2	4	8	16	32	64	128	256
	Розмір матриці										
	1000x1000		0,02316	0,03268	0,03179	0,0276	0,0401	0,0517	0,084	0,1316	0,251
	2500x2500		0,09292	0,10006	0,09847	0,11384	0,1116	0,1384	0,143	0,192	0,3003
	5000x5000		0,25418	0,25518	0,26793	0,31525	0,3504	0.3766	0.395	0.458	0.548
	10000x10000		0,87851	0,96181	1,0085	1,14172	1,197	1,207	1,398	1,432	1,385
	20000x20000		3,4542	3,70934	3,62829	3,71287	5,144	5,173	5.237	5.359	5.489
			час	час	час	час	час	час	час	час	час
Атомарні змінні (не опт)		Кількість потоків	1	2	4	8	16	32	64	128	256
	Розмір матриці										
	1000x1000		0,02452	0,00787	0,01056	0,00842	0,01288	0,02783	0,046	0,1207	0,222
	2500x2500		0,07444	0,03976	0,0379	0,01564	0,01665	0,02503	0,052	0,092	0,212
	5000x5000		0.206617	0,1472	0,06659	0.0422489	0.03512	0.0506	0.064	0.138	0.241
	10000x10000		0,77442	0,49603	0,25586	0,15858	0,09931	0,112	0,1744	0,1826	0,266
	20000x20000		3,00045	1,53008	1,01252	0.485164	0.3422	0.356	0.4009	0.435	0.584
			час	час	час	час	час	час	час	час	час

Матриця 1000x1000



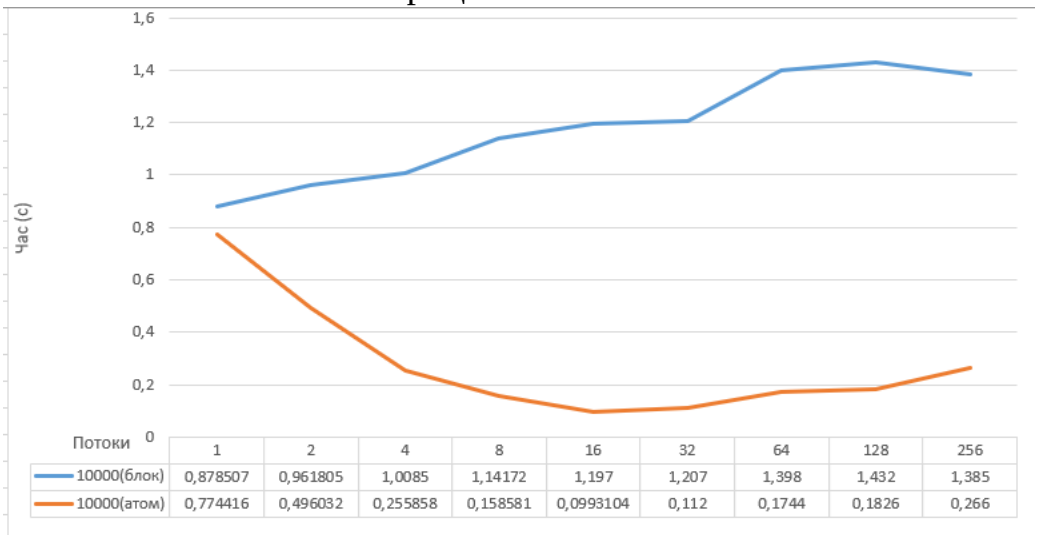
Потоки	1	2	4	8	16	32	64	128	256
1000(блок)	0,0231644	0,0326814	0,0317919	0,0276045	0,0401	0,0517	0,084	0,1316	0,251
1000(атом)	0,0245226	0,0078748	0,0105638	0,0084229	0,0128768	0,02783	0,046	0,1207	0,222

Матриця 2500x2500



Потоки	1	2	4	8	16	32	64	128	256
2500(блок)	0,0929154	0,100059	0,0984669	0,113843	0,1116	0,1384	0,143	0,192	0,3003
2500(атом)	0,0744388	0,0397634	0,0378995	0,0156438	0,0166547	0,02503	0,052	0,092	0,212

Матриця 10000x10000

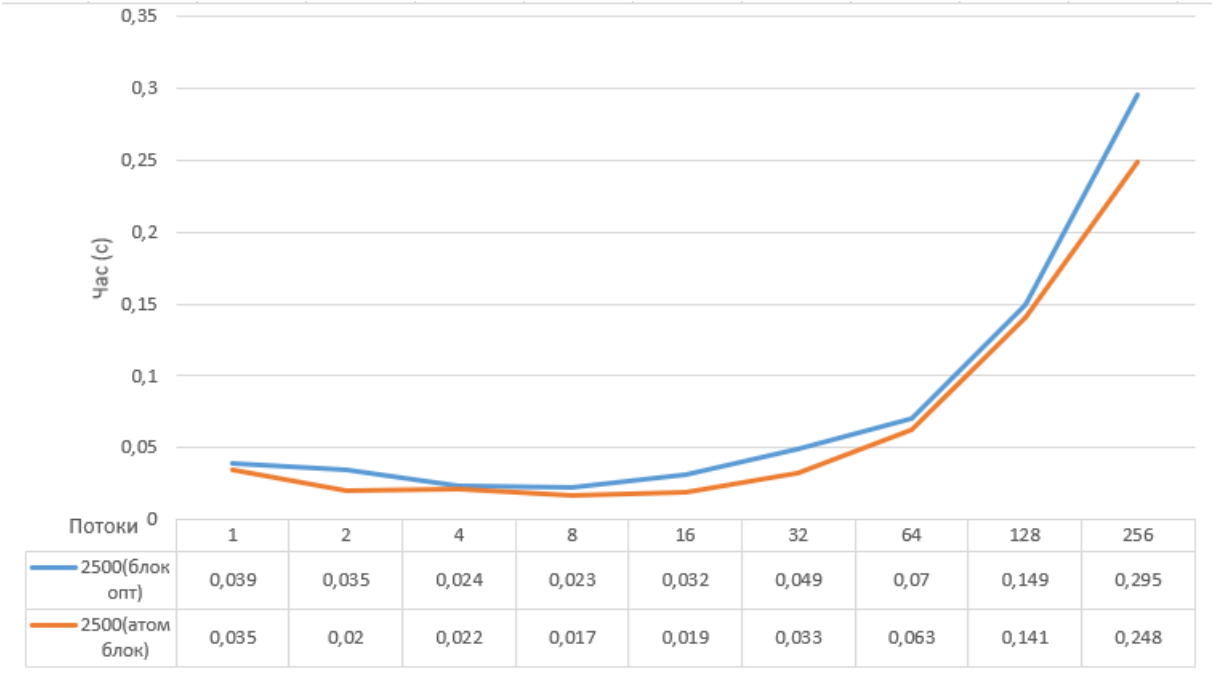


Потоки	1	2	4	8	16	32	64	128	256
10000(блок)	0,878507	0,961805	1,0085	1,14172	1,197	1,207	1,398	1,432	1,385
10000(атом)	0,774416	0,496032	0,255858	0,158581	0,0993104	0,112	0,1744	0,1826	0,266

Графіки та результати при використанні оптімізованих функцій.

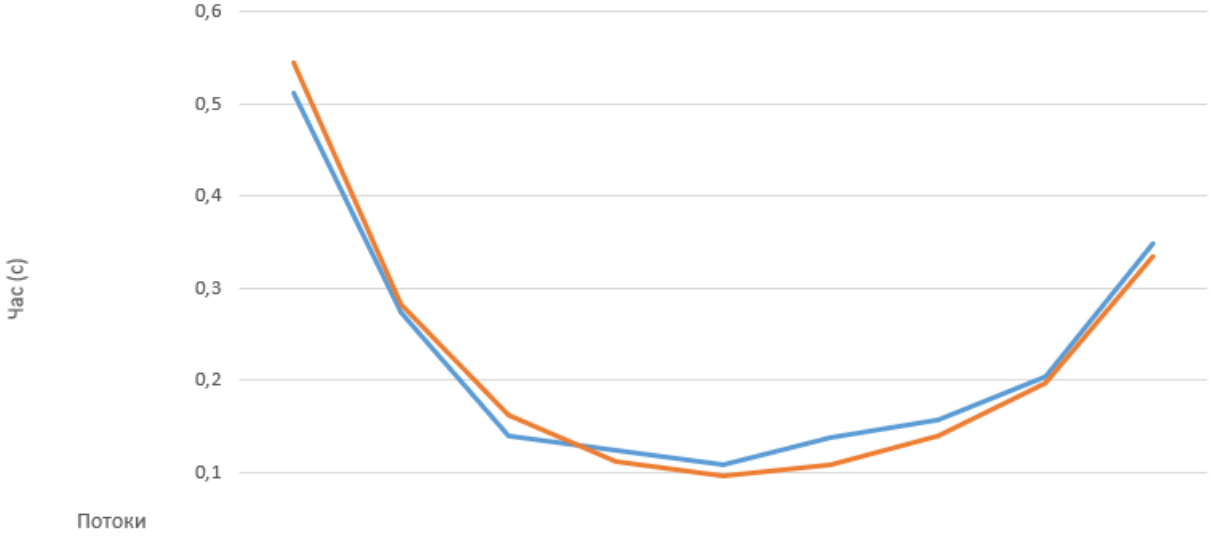
Блокування (опт)		Кількість потоків	1	2	4	8	16	32	64	128	256
	Розмір матриці										
	2500x2500		0,039	0,035	0,024	0,023	0,032	0,049	0,07	0,149	0,295
	10000x10000		0,512	0,273	0,14	0,124	0,109	0,139	0,158	0,205	0,349
			час	час	час	час	час	час	час	час	час
Атомарні змінні (опт)		Кількість потоків	1	2	4	8	16	32	64	128	256
	Розмір матриці										
	2500x2500		0,035	0,02	0,022	0,017	0,019	0,033	0,063	0,141	0,248
	10000x10000		0,544	0,283	0,162	0,112	0,096	0,108	0,14	0,198	0,335
			час	час	час	час	час	час	час	час	час

Матриця 2500x2500



Потоки	1	2	4	8	16	32	64	128	256
2500(блок опт)	0,039	0,035	0,024	0,023	0,032	0,049	0,07	0,149	0,295
2500(атом блок)	0,035	0,02	0,022	0,017	0,019	0,033	0,063	0,141	0,248

Матриця 10000x10000



	1	2	4	8	16	32	64	128	256
10000(блок опт)	0,512	0,273	0,14	0,124	0,109	0,139	0,158	0,205	0,349
10000(атом блок)	0,544	0,283	0,162	0,112	0,096	0,108	0,14	0,198	0,335

Висновок

У ході виконання лабораторної роботи №3 було написано програму підрахунку кількості чисел що кратні 19, та найменше таке число. Було реалізовано обрахунки за допомогою блокуючого примітиву mutex та з використанням атомарних змінних. Було реалізовано 2 методи (оптимзований та не оптимізований для кожного пункту) для порівняння їх між собою. З графіків можна зробити висновок що якщо брати неоптимізовані методи, де ми для мютексу робимо блокування/розблокування на кожній ітерації циклу а для атомарних змінних де ми виконуємо велику кількость атомарних операцій fetch_add та compare_exchange_weak, можемо побачити що по часу методи з атомарними змінними виконуются швидше.
У випадку з оптимізацією обох методів:

- використання глобальних змінних, що дозволяють нам зводити до мінімуму використання мютексів, використовуючи їх лише один раз після обробки всієї частини матриці, яка призначена для конкретного потоку, мютекс блокується тільки один раз для оновлення спільних змінних після завершення обчислень у певному сегменті, замість блокування на кожній ітерації.

- В оптимізованій версії, метод AtomicCompute збирає локальні підрахунки у змінні local_count і local_min для кожного сегменту матриці, який обробляється окремим потоком. Тільки після завершення обходу всього сегменту, здійснюється одноразове атомарне збільшення глобального лічильника count на значення local_count і, за потреби, атомарне оновлення мінімального значення.

З графіків також можемо побачити, що оптимізація обох методів дала значне покращення по часу для м’ютексів і незначий для атомарних змінних.

Додаток 1

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <mutex>
#include <thread>
#include <atomic>
#include <windows.h>

const int DIVISOR = 19;

int** Alloc_Matrix(int size) {
    int** matrix = new int* [size];
    for (int i = 0; i < size; i++) {
        matrix[i] = new int[size];
    }
    return matrix;
}

void Delete_Matrix(int** matrix, int size) {
    for (int i = 0; i < size; i++) {
        delete[] matrix[i];
    }
    delete[] matrix;
}

void Fill_Matrix(int** matrix, int size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            matrix[i][j] = rand() % 100000 + 1;
        }
    }
}

void Print(int count, int min) {
    std::cout << "Кількість чисел що ділятся на 19: " << count << std::endl;
    std::cout << "Найменше число: " << min << std::endl;
}

void SingleThreadSolution(int** matrix, int size) {
    auto start = std::chrono::high_resolution_clock::now();
    int count = 0;
    int min = INT_MAX;

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            int num = matrix[i][j];
```

```
            if (num % DIVISOR == 0) {
                count++;
                if (num < min) {
                    min = num;
                }
            }
        }
    }

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end - start;

    Print(count, min);
    std::cout << "Пройдений час: " << elapsed.count() << " секунд" << std::endl;
}

// не опт
void MutexCompute(int** matrix, int size, int start, int end, int& count, int&
min, std::mutex& mtx) {
    for (int i = start; i < end; i++) {
        for (int j = 0; j < size; j++) {
            if (matrix[i][j] % DIVISOR == 0)
            {
                mtx.lock();
                count++;
                if (matrix[i][j] < min)
                {
                    min = matrix[i][j];
                }
                mtx.unlock();
            }
        }
    }
}

// опт
/*
void MutexCompute(int** matrix, int size, int start, int end, int& count, int&
min, std::mutex& mtx) {
    int local_count = 0;
    int local_min = INT_MAX;
    for (int i = start; i < end; i++) {
        for (int j = 0; j < size; j++) {
            if (matrix[i][j] % DIVISOR == 0) {
                local_count++;
                if (matrix[i][j] < local_min) {
                    local_min = matrix[i][j];
                }
            }
        }
    }
    mtx.lock();
    count += local_count;
    if (local_min < min) {
        min = local_min;
    }
    mtx.unlock();
}
*/

void ParallelComputeMutex(int** matrix, int size, int num_threads) {
    std::vector<std::thread> threads;
    int segment = size / num_threads;
    int count = 0, min = INT_MAX;
    std::mutex mtx;
```

```
    auto start_time = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < num_threads; i++) {
        int start_index = i * segment;
        int end_index;
        if (i == num_threads - 1)
        {
            end_index = size;
        }
        else
        {
            end_index = start_index + segment;
        }
        threads.emplace_back(MutexCompute, matrix, size, start_index, end_index,
std::ref(count), std::ref(min), std::ref(mtx));
    }

    for (auto& thread : threads) {
        thread.join();
    }
    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end_time - start_time;

    Print(count, min);
    std::cout << "Час виконання з мютексами: " << elapsed.count() << " секунд."
<< std::endl;
}

// не опт
void AtomicCompute(int** matrix, int size, int start, int end, std::atomic<int>&
count, std::atomic<int>& minimum) {
    for (int i = start; i < end; i++) {
        for (int j = 0; j < size; j++) {
            if (matrix[i][j] % DIVISOR == 0)
            {
                count.fetch_add(1);
                int current_min = minimum.load();
                do {
                    if (matrix[i][j] >= current_min) {
                        break;
                    }
                } while (!minimum.compare_exchange_weak(current_min, matrix[i]
[j]));
            }
        }
    }
}

// опт
/*
void AtomicCompute(int** matrix, int size, int start, int end, std::atomic<int>&
count, std::atomic<int>& minimum) {
    int local_count = 0;

    for (int i = start; i < end; i++) {
        for (int j = 0; j < size; j++) {
            if (matrix[i][j] % DIVISOR == 0) {
                local_count++;
                int current_min = minimum.load();

                do {
                    if (matrix[i][j] >= current_min) {
                        break;
                    }
```

```
                } while (!minimum.compare_exchange_weak(current_min, matrix[i]
[j]));
            }
        }
    }
    count.fetch_add(local_count);
}
*/

void ParallelComputeAtomic(int** matrix, int size, int num_threads) {
    std::vector<std::thread> threads;
    int segment = size / num_threads;
    std::atomic<int> count(0), minimum(INT_MAX);


    auto start_time = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < num_threads; i++) {
        int start_index = i * segment;
        int end_index;
        if (i == num_threads - 1)
        {
            end_index = size;
        }
        else
        {
            end_index = start_index + segment;
        }
        threads.emplace_back(AtomicCompute, matrix, size, start_index, end_index,
std::ref(count), std::ref(minimum));
    }

    for (auto& thread : threads) {
        thread.join();
    }
    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end_time - start_time;

    Print(count.load(), minimum.load());
    std::cout << "Час виконання з атоміками: " << elapsed.count() << " секунд."
<< std::endl;
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    srand(time(NULL));


    int N;
    std::cout << "Введіть розмір матриці: ";
    std::cin >> N;

    int num_threads;
    std::cout << "Введіть кількість потоків: ";
    std::cin >> num_threads;

    std::cout << std::endl;

    int** matrix = Alloc_Matrix(N);

    Fill_Matrix(matrix, N);
```

```
    std::cout << "Без використання паралелезації: " << std::endl;
    SingleThreadSolution(matrix, N);

     std::cout << std::endl;

     std::cout << "З використанням мютекса: " << std::endl;
     ParallelComputeMutex(matrix, N, num_threads);

     std::cout << std::endl;

     std::cout << "З використанням атоманрих змінних: " << std::endl;
     ParallelComputeAtomic(matrix, N, num_threads);

     Delete_Matrix(matrix, N);

     return 0;
}
```